Compte-rendu — Exercice DevOps et monitoring

Auteur : Pierrick Onchalo — **Date :** 21 octobre 2026 **Dépôt :** github.com/Jhanemba64/voltauto-devops-monitoring (privé)

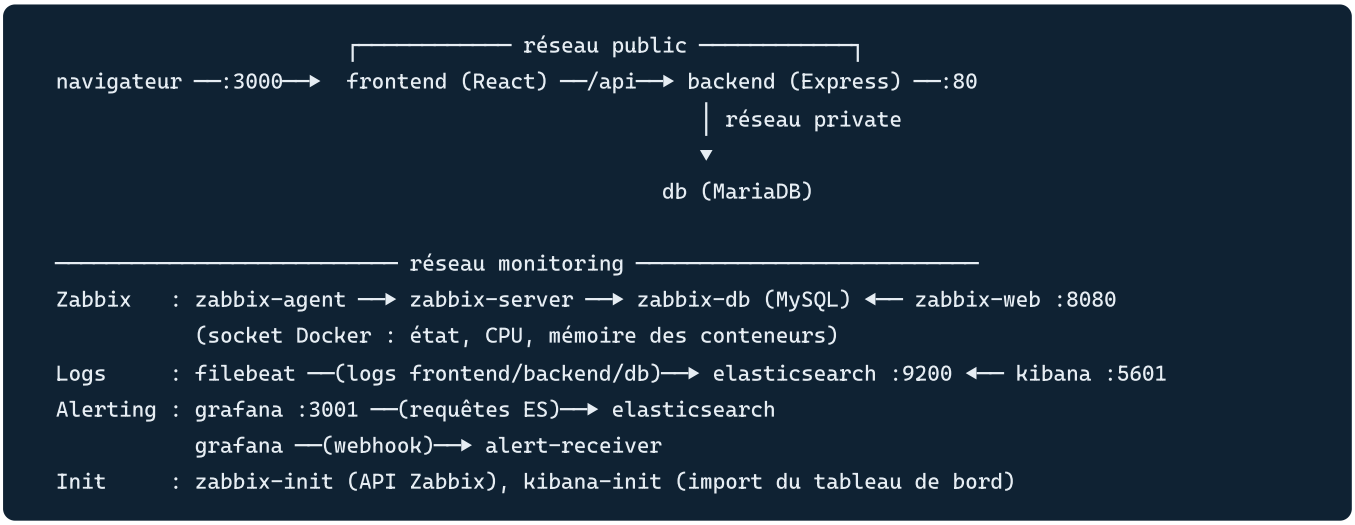
1. Objectif

L'énoncé demande de fournir une architecture web prête à l'emploi sous Docker, puis d'y ajouter une vraie chaîne de supervision, en versionnant le travail selon les bonnes pratiques git :

Étape	Branche	Outil	Rôle
1	`main`	projet `docker/awesome-compose`	application web (frontend, backend, base)
2	`feat/check_infra`	Zabbix	monitoring de l'infrastructure
3	`feat/check_app`	Elasticsearch + Kibana	observabilité de l'application
4	`feat/alerting`	Grafana	alerting avec seuil

L'énoncé cite GitLab. Le travail a été hébergé sur **GitHub** : les *pull requests* y jouent le rôle des *merge requests*.

2. Architecture finale



Toute la pile se lance avec une seule commande :

```
cp .env.example .env      # mots de passe (fichier non versionné)
docker compose up -d --build
```

Toute la configuration est versionnée et appliquée automatiquement : aucune action manuelle dans les interfaces n'est nécessaire.

Service	Image	Accès
frontend / backend / db	build local, `mariadb:10.6.4`	localhost:3000, 127.0.0.1:80
zabbix-server / web / agent	`zabbix/*:alpine-7.0.32` (LTS)	localhost:8080
zabbix-db	`mysql:8.4`	interne
elasticsearch / kibana / filebeat	`8.19.23`	127.0.0.1:9200, 127.0.0.1:5601
grafana	`grafana/grafana:13.2.4`	localhost:3001
alert-receiver	`mendhak/http-https-echo:41`	interne

3. Organisation git

- `main` ne reçoit que l'import initial, puis uniquement des fusions de *pull request*.
- Une branche `feat/...` par étape. Chaque branche contient un commit `feat(...)` pour le code et un commit `docs:` pour le README (Conventional Commits).
- Fusion par *merge commit* (l'historique de la branche reste visible), avec **suppression de la branche** sur GitHub et en local (`gh pr merge --merge --delete-branch`).
- Les secrets ne sont pas versionnés : `.env` est ignoré et `.env.example` sert de modèle.
- `.gitattributes` force les fins de ligne LF, indispensables pour les scripts exécutés dans les conteneurs Linux depuis Windows.
- Un défaut d'affichage trouvé après coup a suivi le même circuit : branche `fix/grafana-echelle-5xx`, puis PR #4. Ce compte-rendu et ses captures arrivent à leur tour par la branche `docs/compte-rendu` (PR #5).

Historique (`git log --oneline --graph`), avant la PR du compte-rendu :

```
*   c2b0318 Merge pull request #4 : échelle du graphique des 5xx (fix/grafana-echelle-5xx)
|\
| * 251b641 fix(grafana): seuil d'alerte visible quand il n'y a aucune erreur 5xx
|/
*   d1f8377 Merge pull request #3 : alerting Grafana (feat/alerting)
|\
| * 84e2a86 docs: section alerting Grafana du README
| * 1df2ba4 feat(alerting): Grafana, tableau de bord avec seuil et alerte sur les erreurs 5xx
|/
*   1e3e78e Merge pull request #2 : observabilité Elasticsearch / Kibana (feat/check_app)
|\
| * 1ceac78 docs: section observabilité Elasticsearch / Kibana du README
| * 023b862 feat(observabilite): logs applicatifs dans Elasticsearch et tableau de bord Kibana
|/
*   9e86b08 Merge pull request #1 : supervision Zabbix (feat/check_infra)
|\
| * 663279c docs: section supervision Zabbix du README
| * d5a30da feat(zabbix): supervision Zabbix (serveur, base MySQL, agent 2, interface web)
|/
* 1c31296 docs: README du projet et normalisation des fins de ligne
* 861b838 chore: import du projet react-express-mysql depuis docker/awesome-compose
```

4. Étape 1 — Projet web prêt à l'emploi

Le projet `react-express-mysql` du dépôt officiel docker/awesome-compose contient les trois couches demandées :

- **frontend** React, servi en mode développement sur le port 3000 ;
- **backend** Node.js / Express. Il logue chaque requête au format *morgan common*, ce qui servira à l'étape 3 ;
- **db** MariaDB, joignable uniquement par le backend (réseau `private`).

Le projet a été importé tel quel (premier commit), testé, puis poussé sur un dépôt GitHub privé créé avec `gh repo create`.

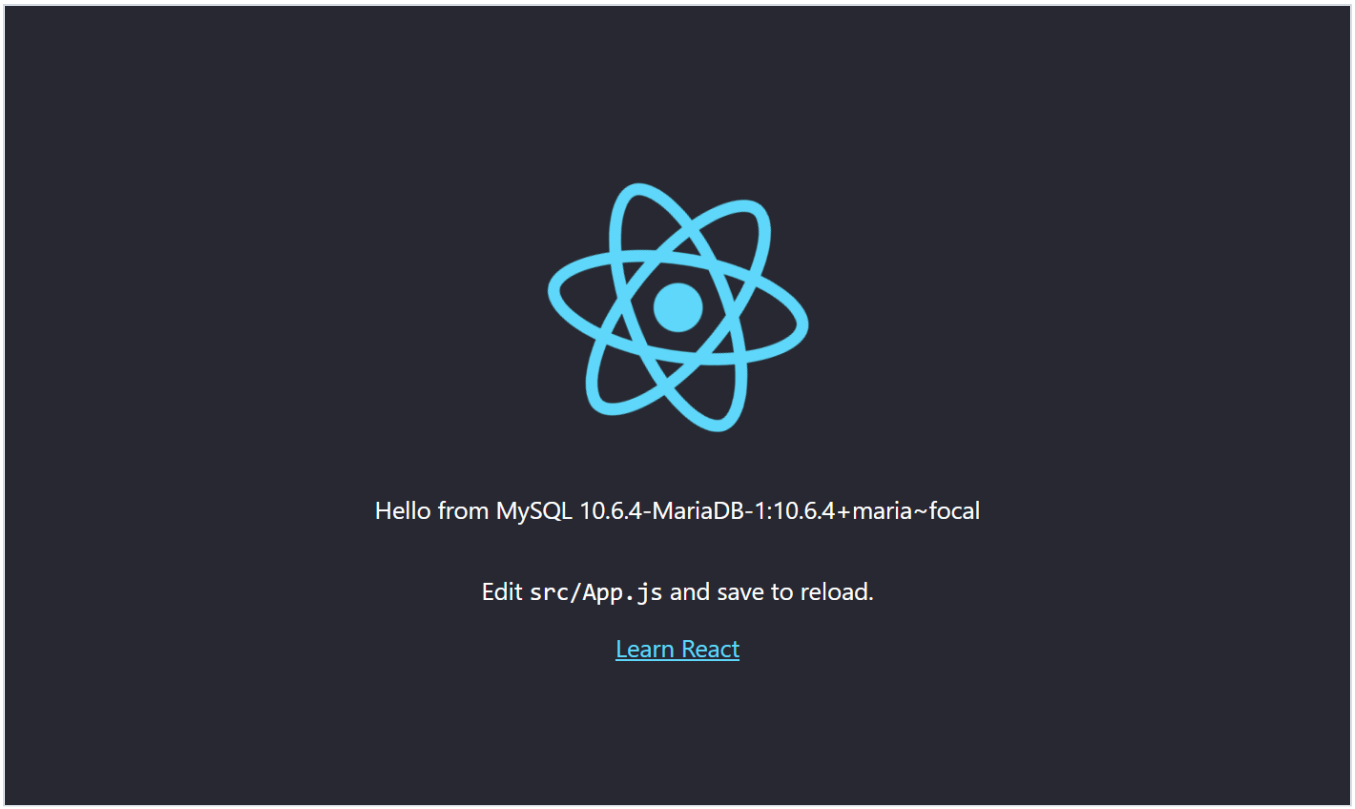


Figure 1 — Application : le frontend affiche la version de la base, lue par le backend

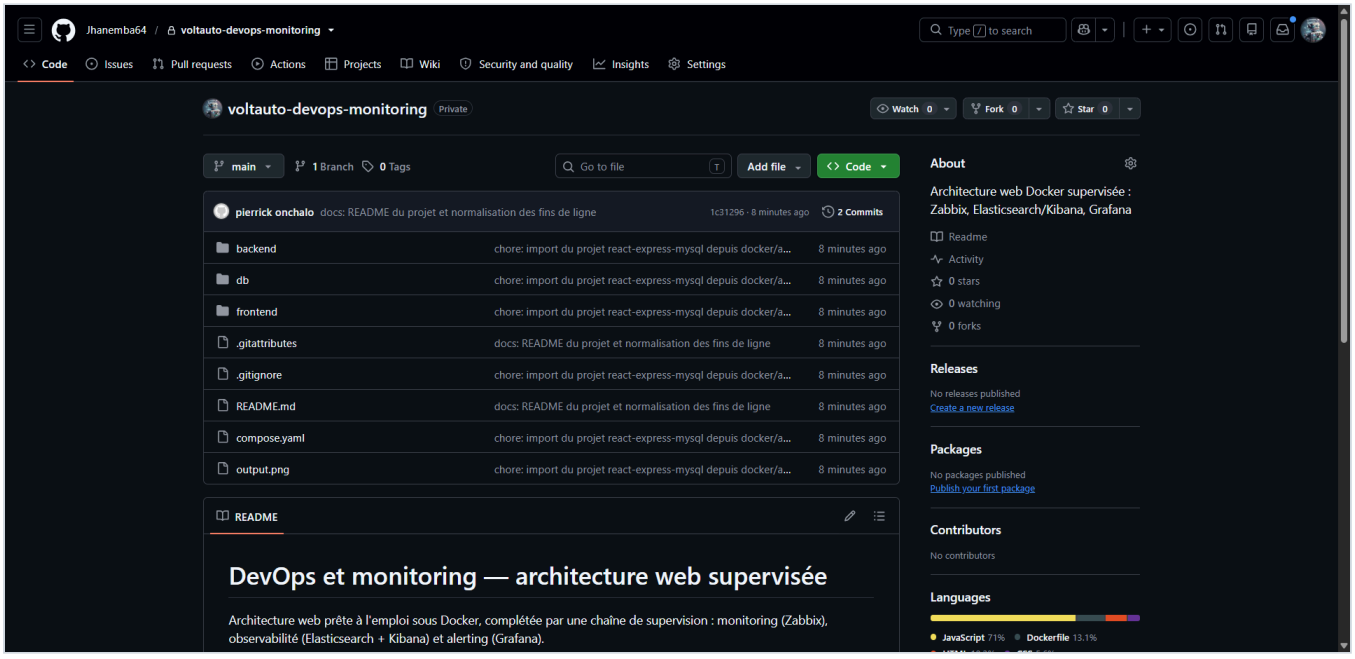


Figure 2 — Dépôt GitHub privé après l'import initial

5. Étape 2 — Supervision avec Zabbix (`feat/check_infra`)

Services ajoutés

Service	Rôle
`zabbix-db`	base MySQL 8.4 de Zabbix (configuration, historique)
`zabbix-server`	collecte, déclencheurs, problèmes
`zabbix-web`	interface web (nginx + PHP)
`zabbix-agent`	agent 2 avec accès en lecture seule au socket Docker

Configuration automatique

Par défaut, le host « Zabbix server » interroge un agent sur `127.0.0.1`. Dans Docker, l'agent est un autre conteneur. Le service ponctuel `zabbix-init` (zabbix/init.sh) appelle donc l'API Zabbix pour :

1. pointer l'interface agent du host vers `zabbix-agent:10050` ;
2. associer le template **Docker by Zabbix agent 2** ;
3. limiter la découverte aux conteneurs du projet, avec la macro `{$DOCKER.LLD.FILTER.CONTAINER.MATCHES} = ^/devops-monitoring-`. Sans ce filtre, tous les conteneurs de la machine seraient remontés.

Le script est idempotent : une seconde exécution ne change rien.

Les captures ci-dessous ont été prises sur la pile complète, après le démarrage à froid de la validation finale (§ 8). Les 12 conteneurs en fonctionnement sont donc supervisés : application, Zabbix, Elastic et Grafana.

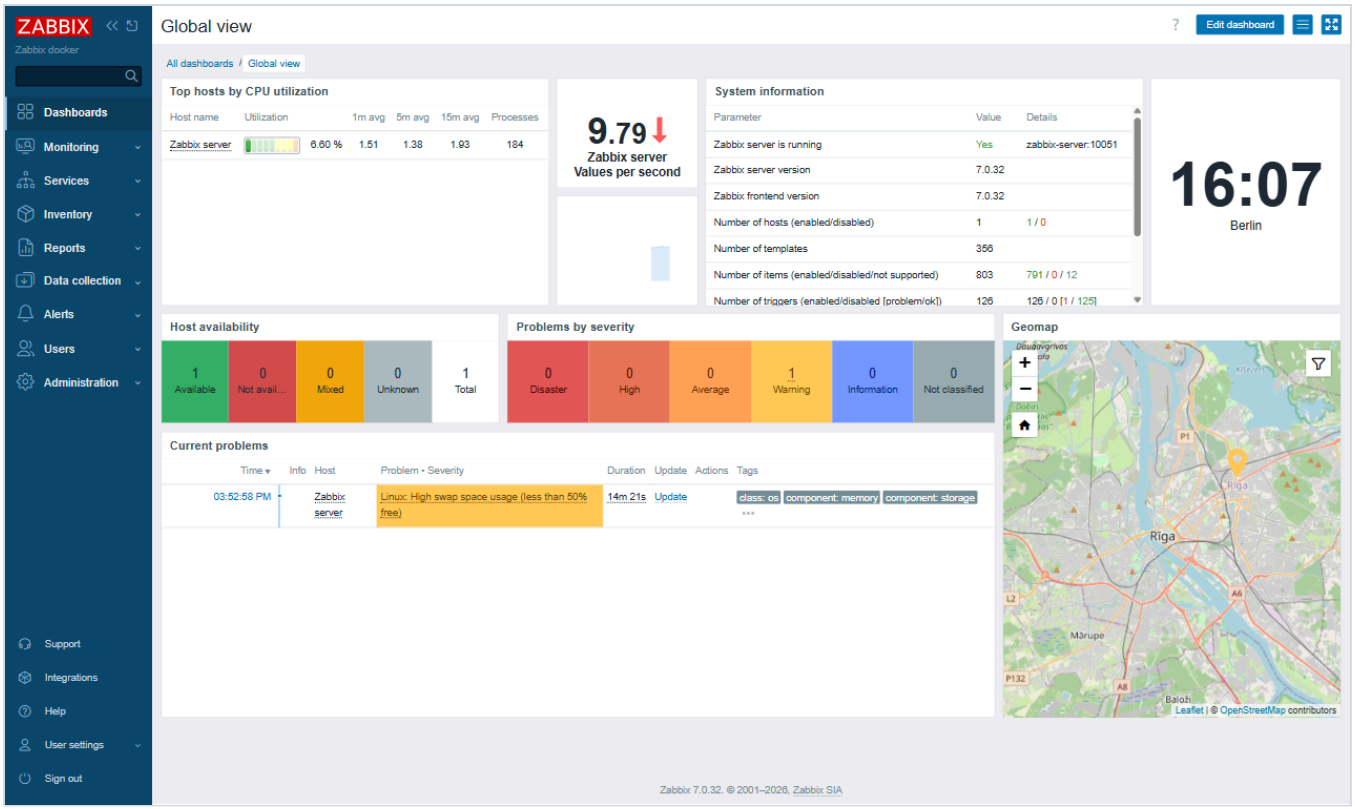


Figure 3 — Tableau de bord Zabbix : host disponible, un avertissement réel sur le swap

Zabbix signale d'ailleurs un vrai problème d'infrastructure, qui n'a pas été provoqué : la VM Docker n'a plus que 15,5 % de swap libre et utilise 73 % de sa mémoire. C'est la conséquence de la charge de la pile complète (Elasticsearch, Kibana, deux bases MySQL…).

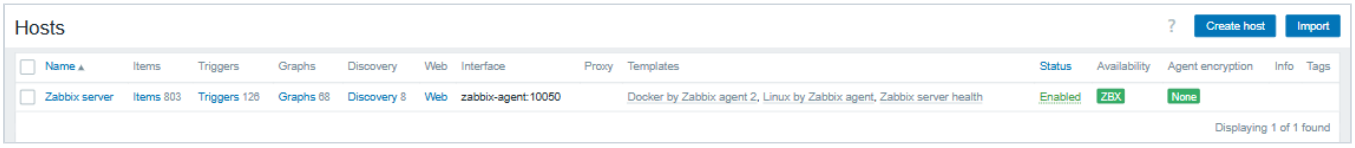


Figure 4 — Host « Zabbix server » supervisé via zabbix-agent:10050 avec les 3 templates (ZBX vert)

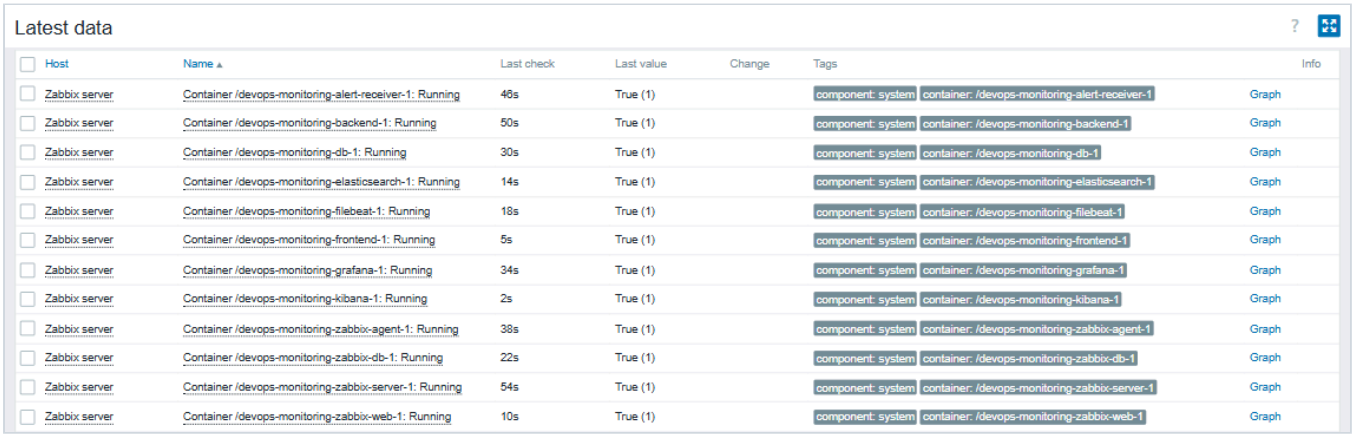


Figure 5 — Les 12 conteneurs du projet sont découverts et à l'état Running

Test

Un arrêt brutal du backend (`docker kill`) déclenche le problème *Container has been stopped with error code* en moins d'une minute (16:09:12). Il se résout automatiquement après

`docker compose start backend` (16:10:12).

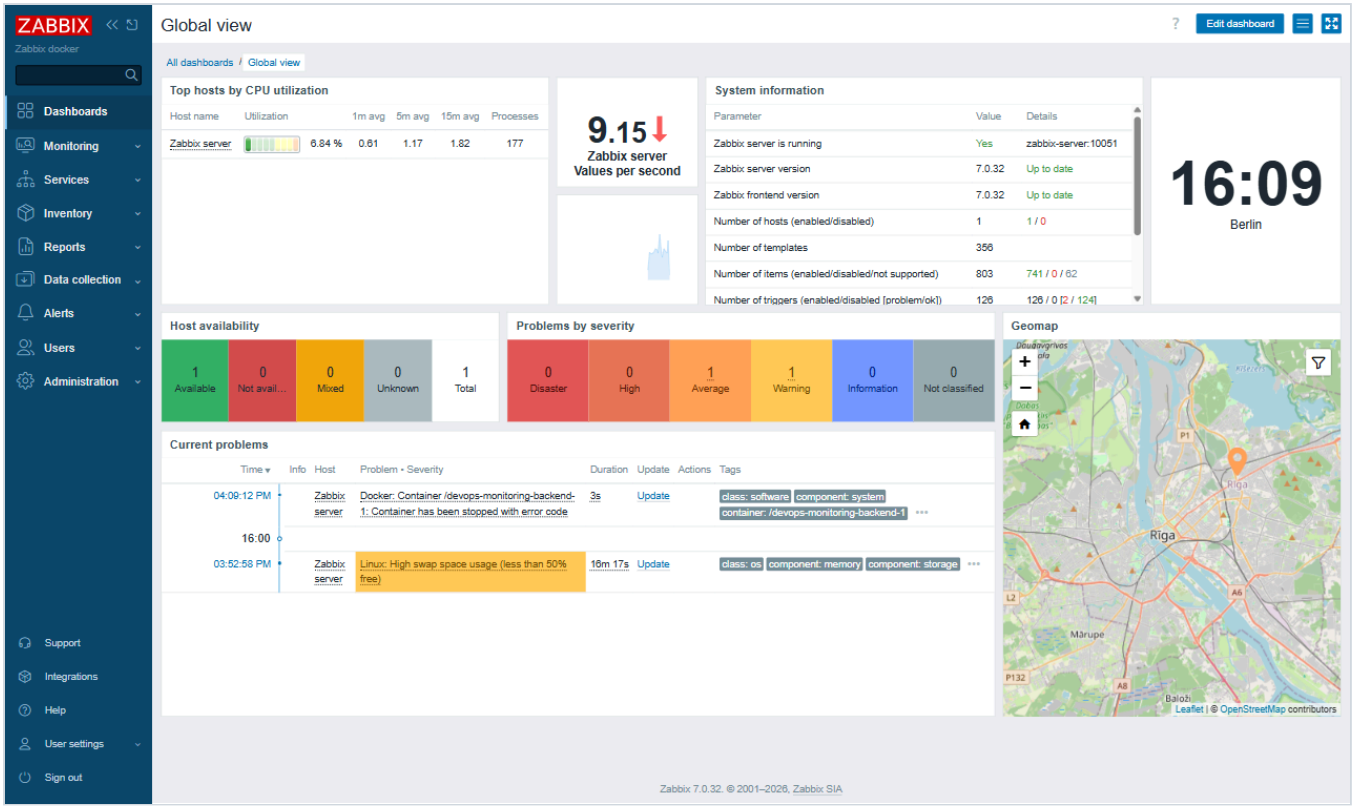


Figure 6 — Problème levé sur l'arrêt du backend

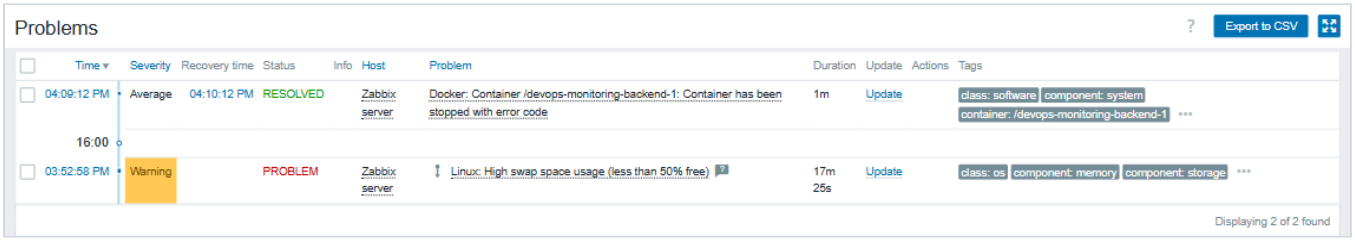


Figure 7 — Problème résolu automatiquement au redémarrage

Commit, pull request, fusion

```
Git Bash — feat/check_infra

pierre@MSI ~/exercice $ git log --stat --format='%C(auto)%h %s' main..feat/check_infra
663279c docs: section supervision Zabbix du README

 README.md | 22 ++++++++++++++++++++-
 1 file changed, 21 insertions(+), 1 deletion(-)
d5a30da feat(zabbix): supervision Zabbix (serveur, base MySQL, agent 2, interface web)

 .env.example    |  3 +++
 .gitignore      |  1 +
 compose.yaml    | 79 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 zabbix/init.sh | 29 ++++++++++++++++++++++
 4 files changed, 112 insertions(+)

pierre@MSI ~/exercice $ git status -sb | head -1
## feat/check_infra...origin/feat/check_infra

pierre@MSI ~/exercice $ git branch -vv
* feat/check_infra 663279c [origin/feat/check_infra] docs: section supervision Zabbix du README
  main            1c31296 [origin/main] docs: README du projet et normalisation des fins de ligne
```

Figure 8 — Commits de la branche feat/check_infra

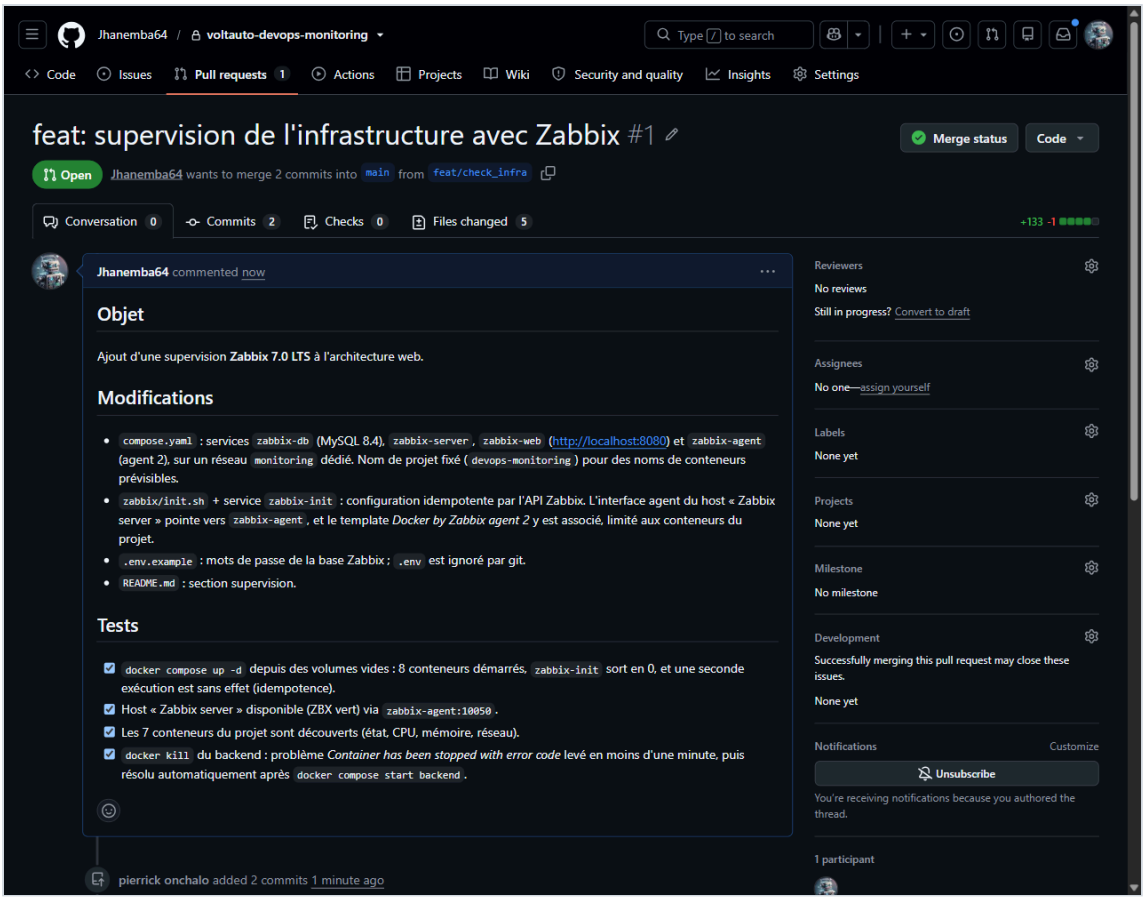


Figure 9 — Pull request #1 : description et tests effectués

Git Bash — merge PR #1

```
pierre@MSI ~/exercice $ gh pr merge 1 --merge --delete-branch --subject 'Merge pull request #1 : supervision Zabbix (feat/check_infra)'
From https://github.com/Jhanemba64/voltauto-devops-monitoring
 * branch            main       -> FETCH_HEAD
   1c31296..9e86b08  main       -> origin/main
Updating 1c31296..9e86b08
Fast-forward
 .env.example   |  3 +++
 .gitignore     |  1 +
 README.md      | 22 +++++++++++++++++-
 compose.yaml   | 79 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 zabbix/init.sh | 29 ++++++++++++++++++++++
 5 files changed, 133 insertions(+), 1 deletion(-)
 create mode 100644 .env.example
 create mode 100644 zabbix/init.sh

pierre@MSI ~/exercice $ git branch -a
* main
  remotes/origin/feat/check_infra
  remotes/origin/main

pierre@MSI ~/exercice $ git log --oneline --graph -6
*   9e86b08 Merge pull request #1 : supervision Zabbix (feat/check_infra)
|\
| * 663279c docs: section supervision Zabbix du README
| * d5a30da feat(zabbix): supervision Zabbix (serveur, base MySQL, agent 2, interface web)
|/
* 1c31296 docs: README du projet et normalisation des fins de ligne
* 861b838 chore: import du projet react-express-mysql depuis docker/awesome-compose

pierre@MSI ~/exercice $ git fetch --prune
From https://github.com/Jhanemba64/voltauto-devops-monitoring
 - [deleted]         (none)     -> origin/feat/check_infra

pierre@MSI ~/exercice $ git ls-remote --heads origin
9e86b08b2b786dd9510b415a842b496af4fd16ee        refs/heads/main

pierre@MSI ~/exercice $ git branch -a
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
```

Figure 10 — Fusion de la PR #1 avec suppression de la branche

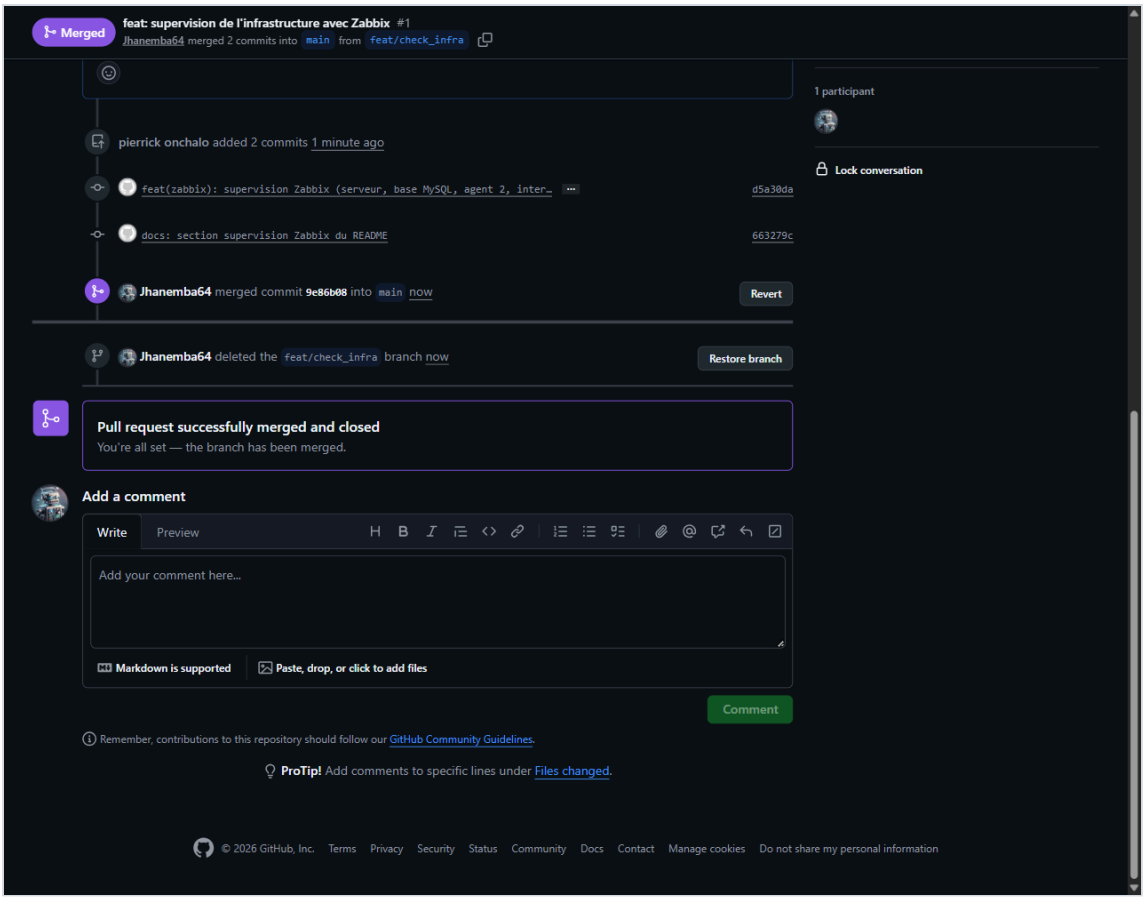


Figure 11 — PR #1 fusionnée, branche feat/check_infra supprimée sur GitHub

6. Étape 3 — Observabilité avec Elasticsearch et Kibana (`feat/check_app`)

Services ajoutés

Elasticsearch et Kibana ne collectent rien par eux-mêmes. Un collecteur, **Filebeat**, a donc été ajouté pour y envoyer les logs de l'application.

Service	Rôle
`elasticsearch`	stockage et recherche des logs (nœud unique, heap de 512 Mo)
`kibana`	exploration et tableaux de bord
`filebeat`	collecte des logs des conteneurs `frontend`, `backend` et `db`
`kibana-init`	import automatique du tableau de bord

Exploitation des logs

filebeat/filebeat.yml découpe chaque ligne de log HTTP du backend en champs structurés (ECS) :

```
::ffff:192.168.48.1 - - [21/Oct/2026:12:58:24 +0000] "GET /inexistant HTTP/1.1" 404 149
  → http.request.method = GET, url.original = /inexistant, http.response.status_code = 404
```

On peut ainsi filtrer et agréger par code retour (Discover, Lens, Grafana).

Tableau de bord « Surveillance de l'application »

Le tableau de bord (kibana/dashboard.ndjson) affiche :

- quatre indicateurs : requêtes HTTP, erreurs 4xx, erreurs 5xx, lignes d'erreur du backend ;
- les requêtes par code retour dans le temps, et leur répartition ;
- le volume de logs par service ;
- les dernières erreurs du backend (stderr).

Pour le tester, la base a été arrêtée 40 secondes sous trafic (15:16:49 → 15:17:31). Le tableau de bord fait alors apparaître les erreurs 500 et les traces `ECONNREFUSED ... :3306` du backend.

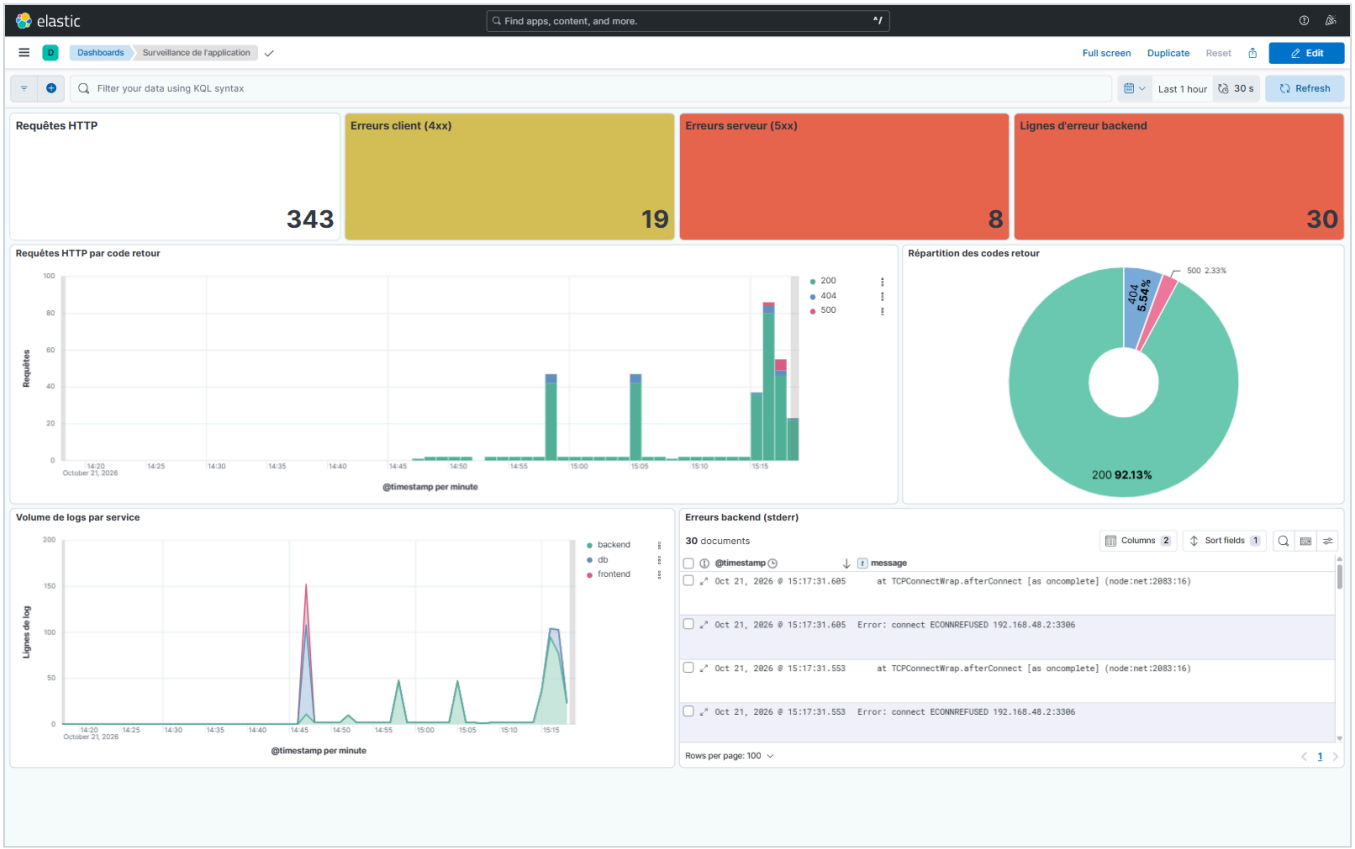


Figure 12 — Tableau de bord Kibana après la panne simulée de la base

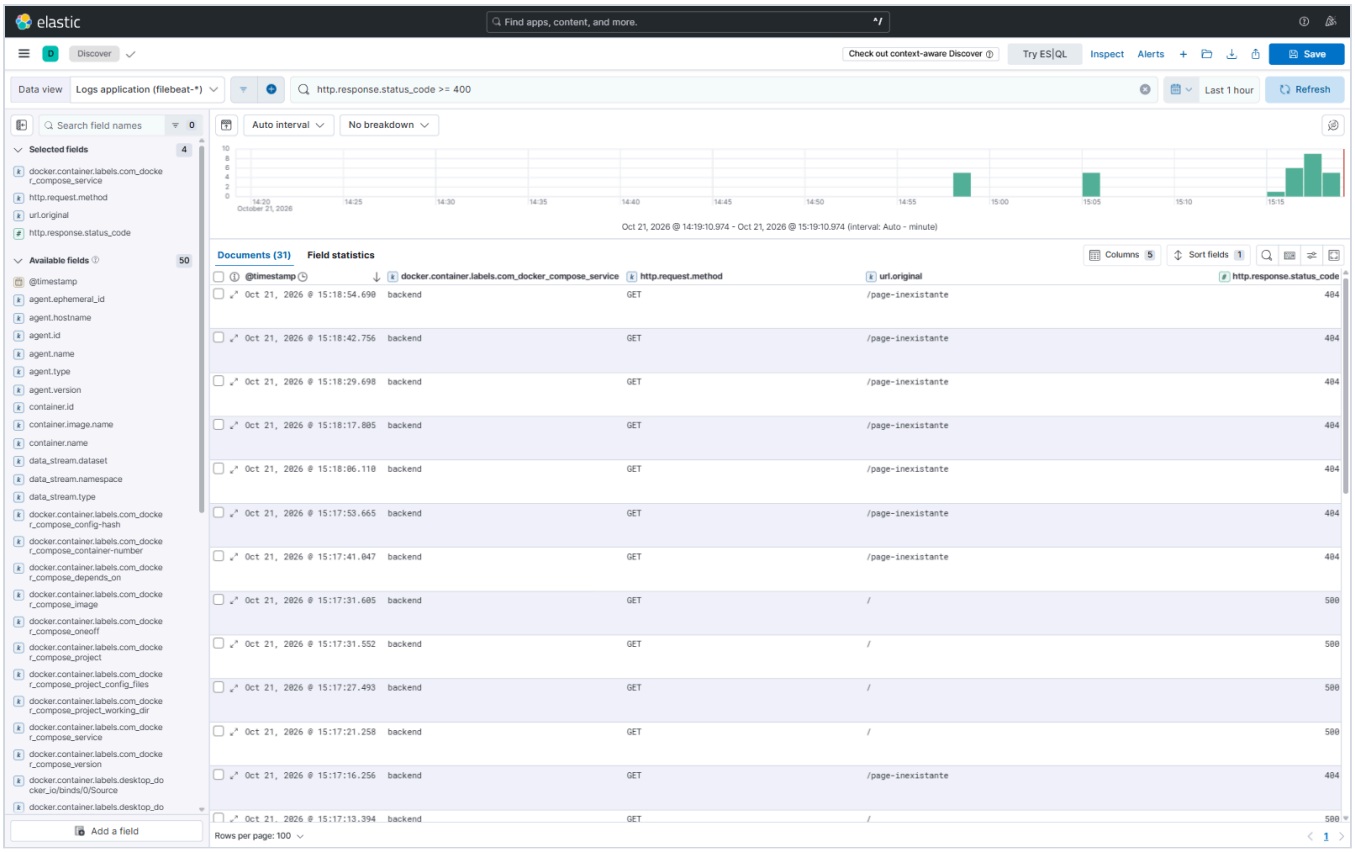


Figure 13 — Discover : requêtes en erreur (404, 500) avec les champs extraits

Commit, pull request, fusion

```
Git Bash — feat/check_app

pierre@MSI ~/exercice $ git log --stat --format='%C(auto)%h %s' main..feat/check_app
1ceac78 docs: section observabilité Elasticsearch / Kibana du README

 README.md | 24 ++++++++++++++++++++++-
 1 file changed, 23 insertions(+), 1 deletion(-)
023b862 feat(observabilite): logs applicatifs dans Elasticsearch et tableau de bord Kibana

 compose.yaml            | 61 +++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 filebeat/filebeat.yml   | 30 ++++++++++++++++++++++++++++++
 kibana/dashboard.ndjson | 10 ++++++++
 kibana/init.sh          | 13 +++++++++++++
 4 files changed, 114 insertions(+)

pierre@MSI ~/exercice $ git status -sb | head -1
## feat/check_app...origin/feat/check_app
```

Figure 14 — Commits de la branche feat/check_app

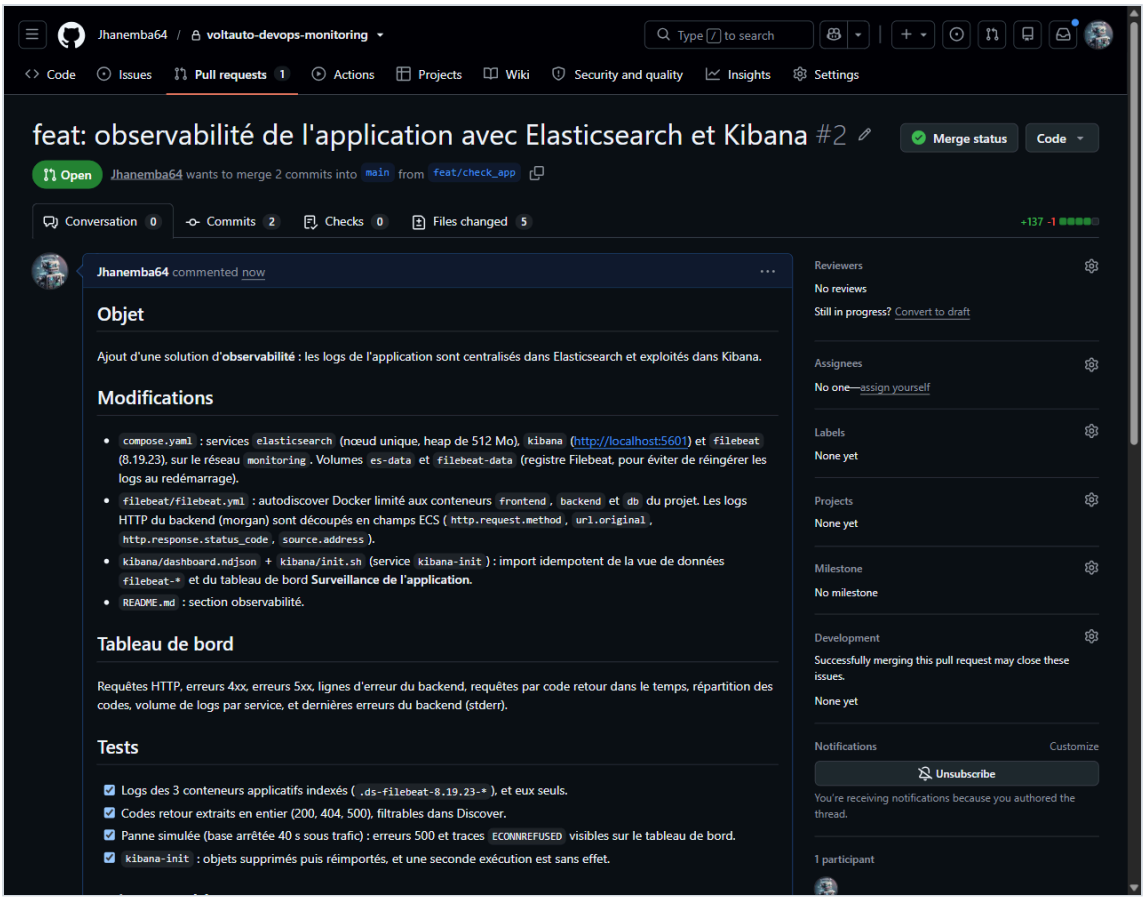


Figure 15 — Pull request #2

```
Git Bash — merge PR #2

pierre@MSI ~/exercice $ gh pr merge 2 --merge --delete-branch --subject 'Merge pull request #2 : observabilité Elasticsearch / Kibana
(feat/check_app)'
From https://github.com/Jhanemba64/voltauto-devops-monitoring
 * branch            main       -> FETCH_HEAD
   9e86b08..1e3e78e  main       -> origin/main
Updating 9e86b08..1e3e78e
Fast-forward
 README.md              | 24 ++++++++++++++++++-
 compose.yaml           | 61 ++++++++++++++++++++++++++++++++++++++++++++++++++
 filebeat/filebeat.yml  | 30 ++++++++++++++++++++++
 kibana/dashboard.ndjson | 10 ++++++++
 kibana/init.sh         | 13 ++++++++++
 5 files changed, 137 insertions(+), 1 deletion(-)
 create mode 100644 filebeat/filebeat.yml
 create mode 100644 kibana/dashboard.ndjson
 create mode 100644 kibana/init.sh

pierre@MSI ~/exercice $ git fetch --prune
From https://github.com/Jhanemba64/voltauto-devops-monitoring
 - [deleted]         (none)     -> origin/feat/check_app

pierre@MSI ~/exercice $ git ls-remote --heads origin
1e3e78ef4a01129128a3dff9280dfa4afaf55059        refs/heads/main

pierre@MSI ~/exercice $ git branch -a
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main

pierre@MSI ~/exercice $ git log --oneline --graph -9
*   1e3e78e Merge pull request #2 : observabilité Elasticsearch / Kibana (feat/check_app)
|\
| * 1ceac78 docs: section observabilité Elasticsearch / Kibana du README
| * 023b862 feat(observabilite): logs applicatifs dans Elasticsearch et tableau de bord Kibana
|/
*   9e86b08 Merge pull request #1 : supervision Zabbix (feat/check_infra)
|\
| * 663279c docs: section supervision Zabbix du README
| * d5a30da feat(zabbix): supervision Zabbix (serveur, base MySQL, agent 2, interface web)
|/
* 1c31296 docs: README du projet et normalisation des fins de ligne
* 861b838 chore: import du projet react-express-mysql depuis docker/awesome-compose
```

Figure 16 — Fusion de la PR #2 avec suppression de la branche

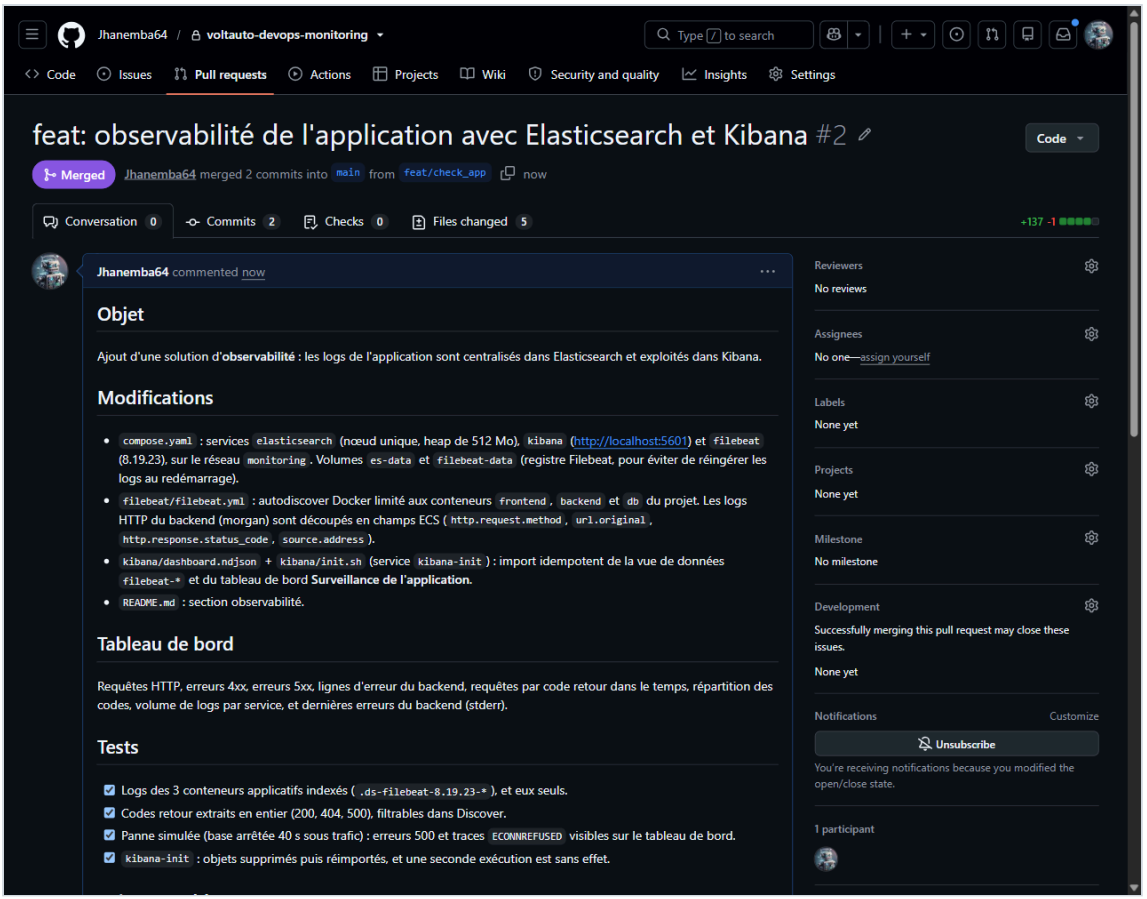


Figure 17 — PR #2 fusionnée, branche feat/check_app supprimée

7. Étape 4 — Alerting avec Grafana (`feat/alerting`)

Services ajoutés

Service	Rôle
`grafana`	tableau de bord avec seuil, règle d'alerte, notifications (port 3001 : le 3000 est pris par le frontend)
`alert-receiver`	récepteur webhook qui simule l'outil de notification (Slack, Teams, mail…) et journalise chaque alerte

Grafana est entièrement **provisionné par fichiers** (grafana/) :

- **source de données** Elasticsearch (`filebeat-*`) ;
- **tableau de bord « Alerting application »** : graphique des erreurs 5xx par minute avec un **seuil à 5** (ligne pointillée, zone rouge, barres rouges au-delà), compteur sur la fenêtre d'alerte, état de l'alerte, requêtes par code retour ;
- **règle « Erreurs 5xx backend »**, en trois étapes :
  - A : nombre de réponses 5xx sur 2 minutes ;
  - B : somme de A ;

- C : seuil B > 5.

  Elle est évaluée toutes les 30 s et ne se déclenche qu'après 30 s au-dessus du seuil, ce qui évite les fausses alertes ;

- **point de contact** webhook vers `alert-receiver`, avec sa politique de notification.

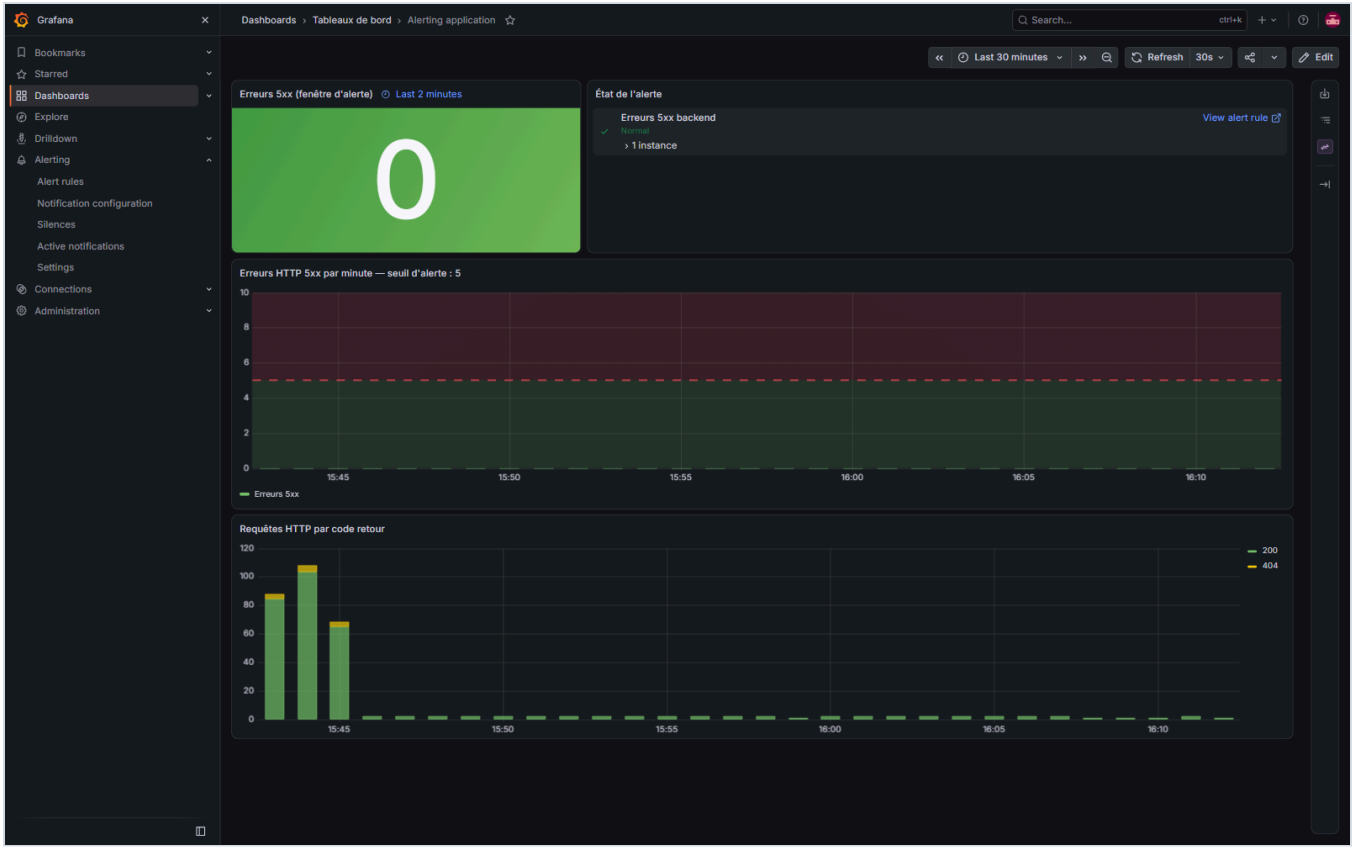


Figure 18 — Tableau de bord Grafana à l'état normal, seuil à 5 en pointillés

Test de bout en bout

La base a été arrêtée sous trafic à 15:34:27. Le backend renvoie alors des erreurs 500 et l'alerte suit le cycle attendu :

Heure	Événement
15:34:27	`docker compose stop db`
15:35:34	règle **Pending** (seuil dépassé)
15:36:01	règle **Firing** (dépassement maintenu 30 s)
15:36:40	notification `FIRING` reçue par le webhook : « 15 réponses 5xx sur les 2 dernières minutes »
15:36:44	`docker compose start db`
15:38:31	règle revenue à **Normal**
15:38:43	notification `RESOLVED` reçue



Figure 19 — Alerte déclenchée : compteur rouge, barres au-dessus du seuil, état Firing

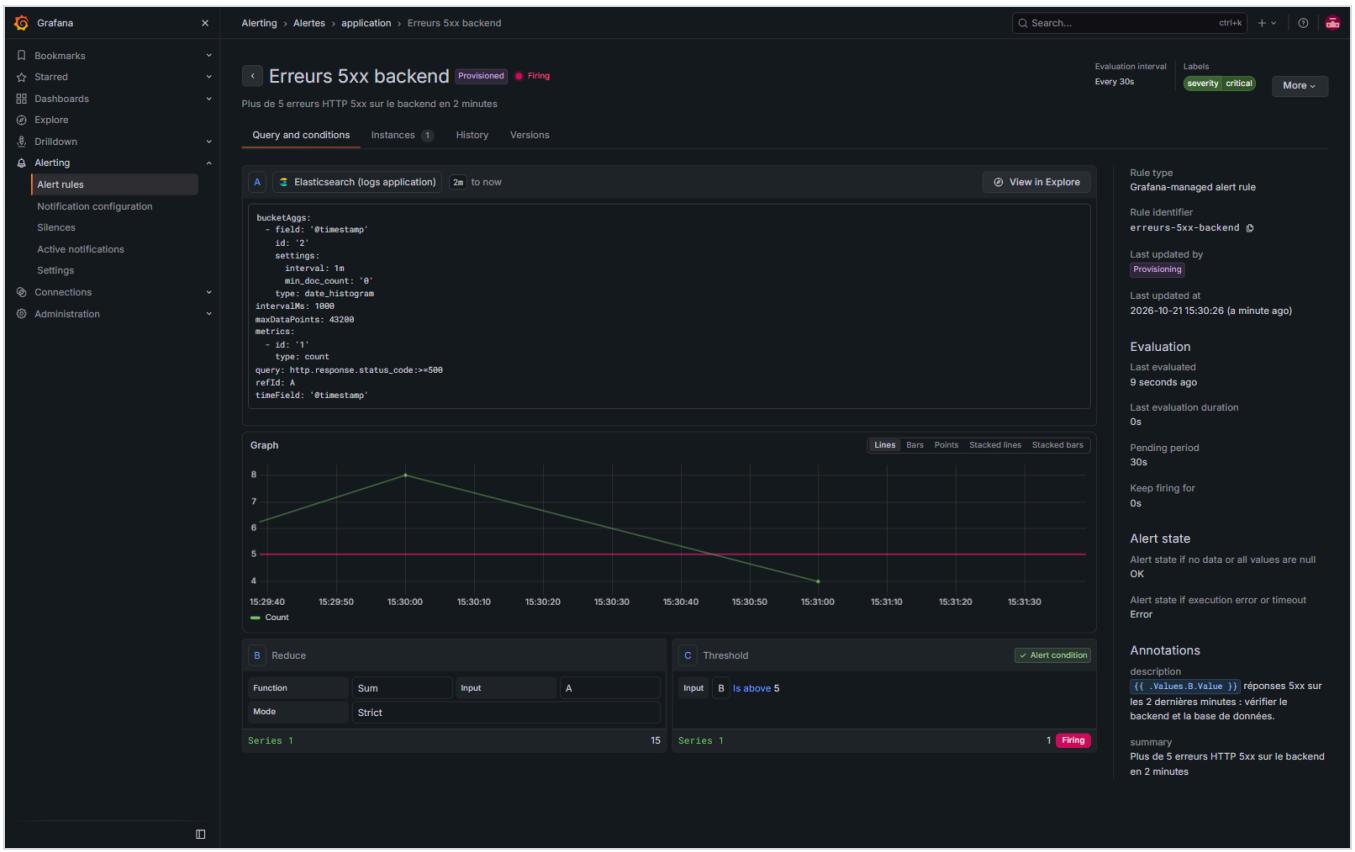


Figure 20 — Détail de la règle : requête A, réduction B (somme = 15), seuil C > 5

Git Bash — notifications reçues par le webhook

```
pierre@MSI ~/exercice $ docker compose logs alert-receiver | grep "POST /grafana"
alert-receiver-1  | ::ffff:192.168.96.13 - - [21/Oct/2026:13:29:40 +0000] "POST /grafana HTTP/1.1" 200 5587 "-" "Grafana"
alert-receiver-1  | ::ffff:192.168.96.13 - - [21/Oct/2026:13:34:36 +0000] "POST /grafana HTTP/1.1" 200 5455 "-" "Grafana"
alert-receiver-1  | ::ffff:192.168.96.13 - - [21/Oct/2026:13:36:40 +0000] "POST /grafana HTTP/1.1" 200 5467 "-" "Grafana"
alert-receiver-1  | ::ffff:192.168.96.13 - - [21/Oct/2026:13:38:43 +0000] "POST /grafana HTTP/1.1" 200 5455 "-" "Grafana"

pierre@MSI ~/exercice $ docker compose logs alert-receiver --no-log-prefix | python notifs.py | tail -12   # 2 dernières notifications
(heures UTC)
POST /grafana  ->  status=firing  title="[FIRING:1] Erreurs 5xx backend Alertes (critical)"
   [FIRING] Erreurs 5xx backend (severity=critical)
      début : 2026-10-21T13:36:00Z   fin : 0001-01-01T00:00:00Z
      Plus de 5 erreurs HTTP 5xx sur le backend en 2 minutes
      15 réponses 5xx sur les 2 dernières minutes : vérifier le backend et la base de données.
      lien : http://localhost:3001/alerting/grafana/erreurs-5xx-backend/view
POST /grafana  ->  status=resolved  title="[RESOLVED] Erreurs 5xx backend Alertes (critical)"
   [RESOLVED] Erreurs 5xx backend (severity=critical)
      début : 2026-10-21T13:36:00Z   fin : 2026-10-21T13:38:30Z
      Plus de 5 erreurs HTTP 5xx sur le backend en 2 minutes
      4 réponses 5xx sur les 2 dernières minutes : vérifier le backend et la base de données.
      lien : http://localhost:3001/alerting/grafana/erreurs-5xx-backend/view
```

Figure 21 — Notifications reçues par le webhook : déclenchement puis résolution

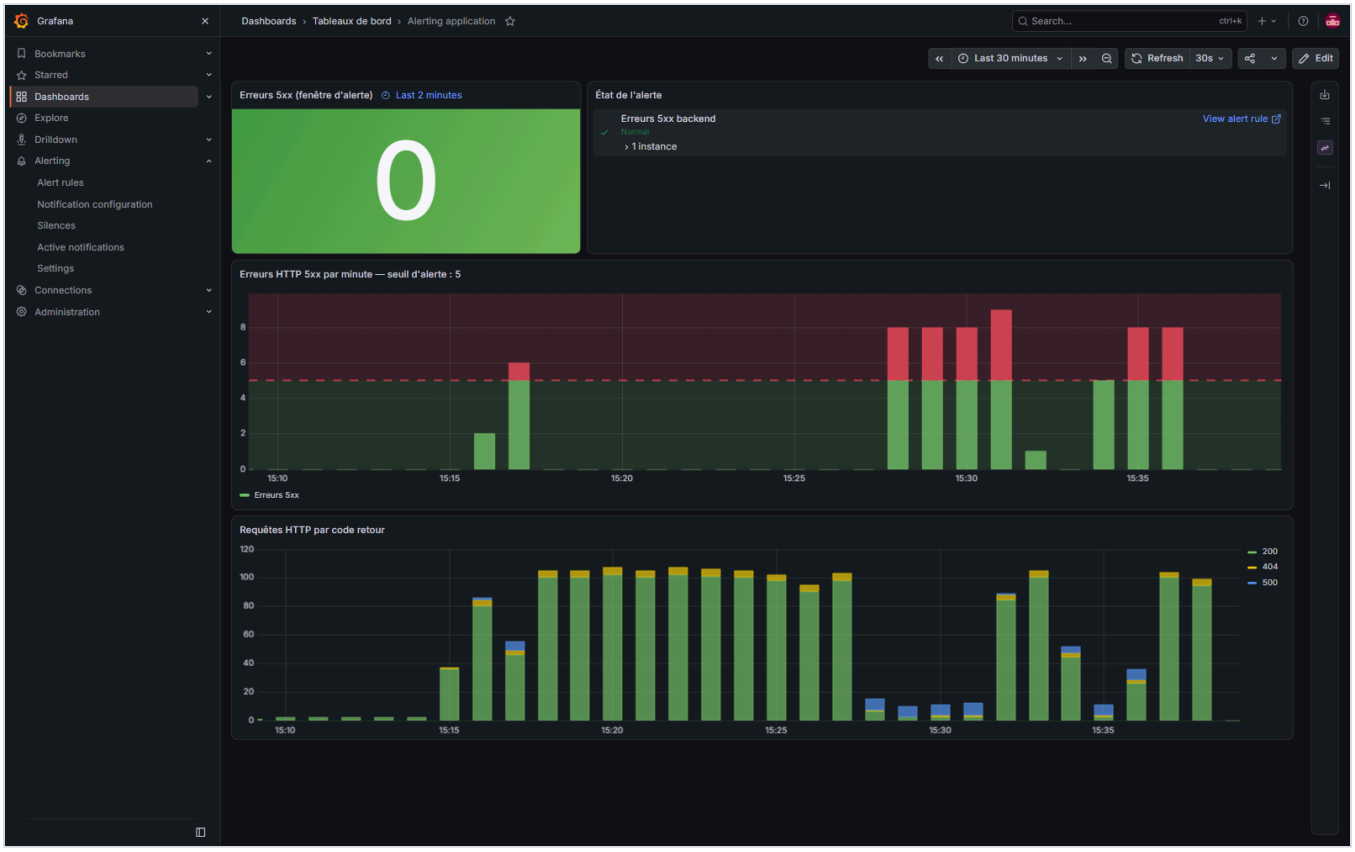


Figure 22 — Retour à la normale après le redémarrage de la base

Commit, pull request, fusion

```
Git Bash — feat/alerting

pierre@MSI ~/exercice $ git log --stat --format='%C(auto)%h %s' main..feat/alerting
84e2a86 docs: section alerting Grafana du README

 README.md | 27 +++++++++++++++++++++++++--
 1 file changed, 25 insertions(+), 2 deletions(-)
1df2ba4 feat(alerting): Grafana, tableau de bord avec seuil et alerte sur les erreurs 5xx

 .env.example                                  |   1 +
 compose.yaml                                  |  30 +++++
 grafana/dashboards/alerting-application.json  | 141 ++++++++++++++++++++++
 grafana/provisioning/alerting/alerting.yml    |  66 ++++++++++
 grafana/provisioning/dashboards/dashboards.yml |   9 ++
 grafana/provisioning/datasources/elasticsearch.yml |  13 ++
 6 files changed, 260 insertions(+)

pierre@MSI ~/exercice $ git status -sb | head -1
## feat/alerting...origin/feat/alerting
```

Figure 23 — Commits de la branche feat/alerting

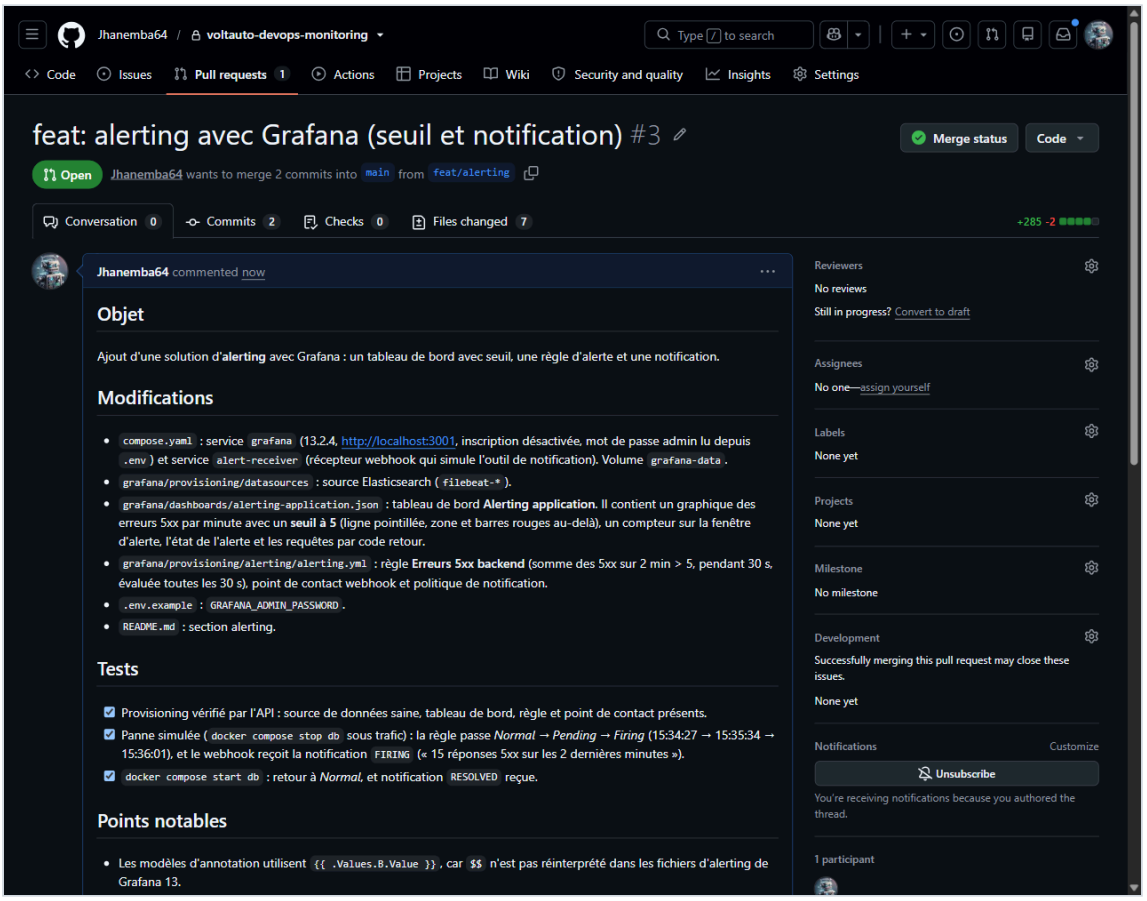


Figure 24 — Pull request #3



Figure 25 — Fusion de la PR #3 et historique complet

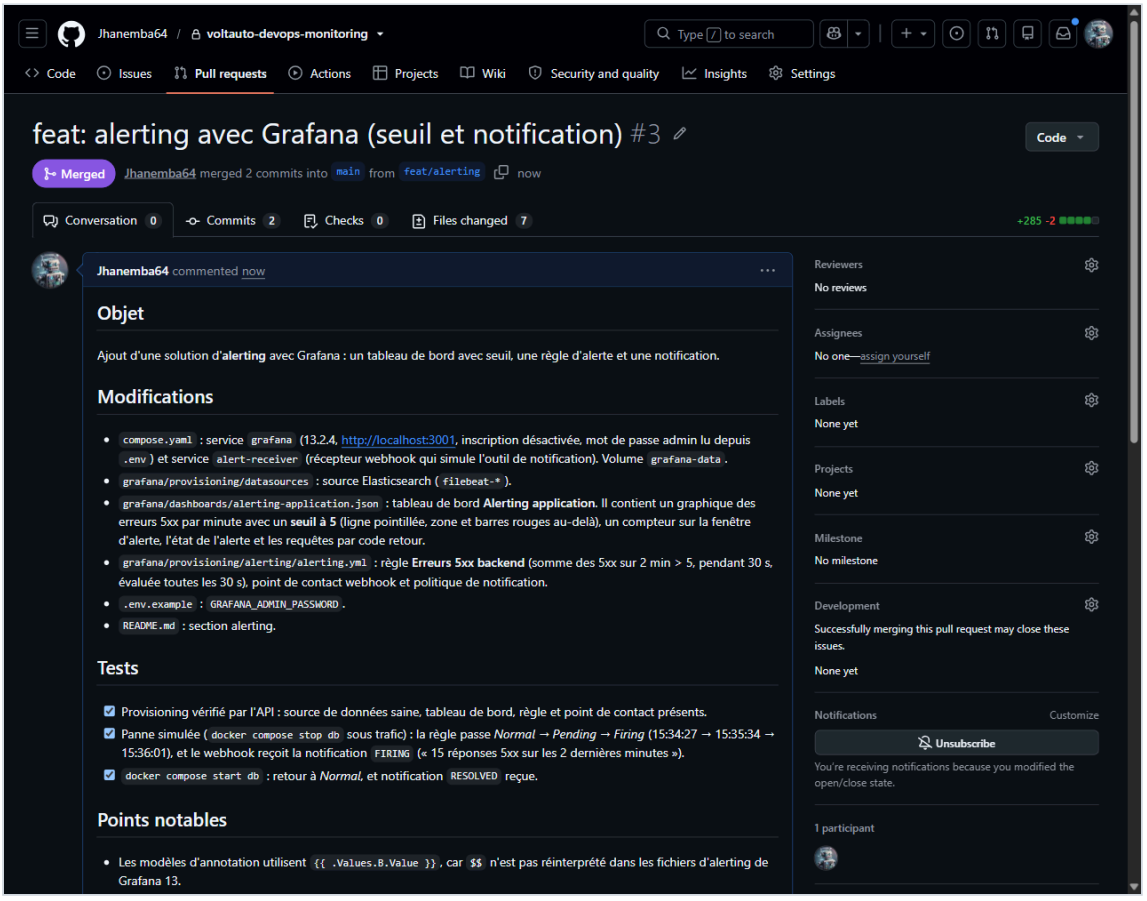


Figure 26 — PR #3 fusionnée, branche feat/alerting supprimée

8. Validation finale

Pour vérifier que `main` est bien prêt à l'emploi, toute la pile a été détruite avec ses volumes (`docker compose down -v`), puis relancée depuis zéro. Les 14 services démarrent, les deux tâches d'initialisation se terminent sans erreur (code 0), l'application répond, les logs arrivent dans Elasticsearch et Grafana est opérationnel.

Git Bash — démarrage à froid de la pile complète

```
pierre@MSI ~/exercice $ docker compose down -v && docker compose up -d --build   # démarrage à froid depuis main
(21 ressources supprimées, pile recréée en 73 s)

pierre@MSI ~/exercice $ docker compose ps -a --format "table {{.Service}}\t{{.Status}}\t{{.Ports}}"
SERVICE          STATUS                          PORTS
alert-receiver   Up 3 minutes                    8080/tcp, 8443/tcp
backend          Up 3 minutes (healthy)          0.0.0.0:80->80/tcp, 0.0.0.0:9229-9230->9229-9230/tcp
db               Up 3 minutes                    3306/tcp
elasticsearch    Up 3 minutes (healthy)          127.0.0.1:9200->9200/tcp, 9300/tcp
filebeat         Up 3 minutes
frontend         Up 3 minutes                    0.0.0.0:3000->3000/tcp
grafana          Up 3 minutes                    0.0.0.0:3001->3000/tcp
kibana           Up 3 minutes                    127.0.0.1:5601->5601/tcp
kibana-init      Exited (0) About a minute ago
zabbix-agent     Up 3 minutes                    10050/tcp, 31999/tcp
zabbix-db        Up 3 minutes (healthy)          3306/tcp, 33060/tcp
zabbix-init      Exited (0) About a minute ago
zabbix-server    Up 3 minutes                    0.0.0.0:10051->10051/tcp
zabbix-web       Up 3 minutes (healthy)          0.0.0.0:8080->8080/tcp, 8443/tcp

pierre@MSI ~/exercice $ docker compose logs --no-log-prefix zabbix-init kibana-init | grep -E "Host|importé"
Host « Zabbix server » (10084) : agent zabbix-agent:10050 + template Docker (10318), conteneurs ^/devops-monitoring-
Tableau de bord importé : http://localhost:5601/app/dashboards#/view/dashboard-surveillance-app

pierre@MSI ~/exercice $ curl -s http://localhost:3000/api/
{"message":"Hello from MySQL 10.6.4-MariaDB-1:10.6.4+maria~focal"}

pierre@MSI ~/exercice $ curl -s 'http://127.0.0.1:9200/filebeat-*/_count?q=url.original:*'
{"count":355,"_shards":{"total":1,"successful":1,"skipped":0,"failed":0}}

pierre@MSI ~/exercice $ curl -s http://localhost:3001/api/health
{"database":"ok","version":"13.2.4","commit":"0e79aef9d91abf3bdadae9af15763029072fb45b"}
```

Figure 27 — Démarrage à froid de la pile complète depuis main

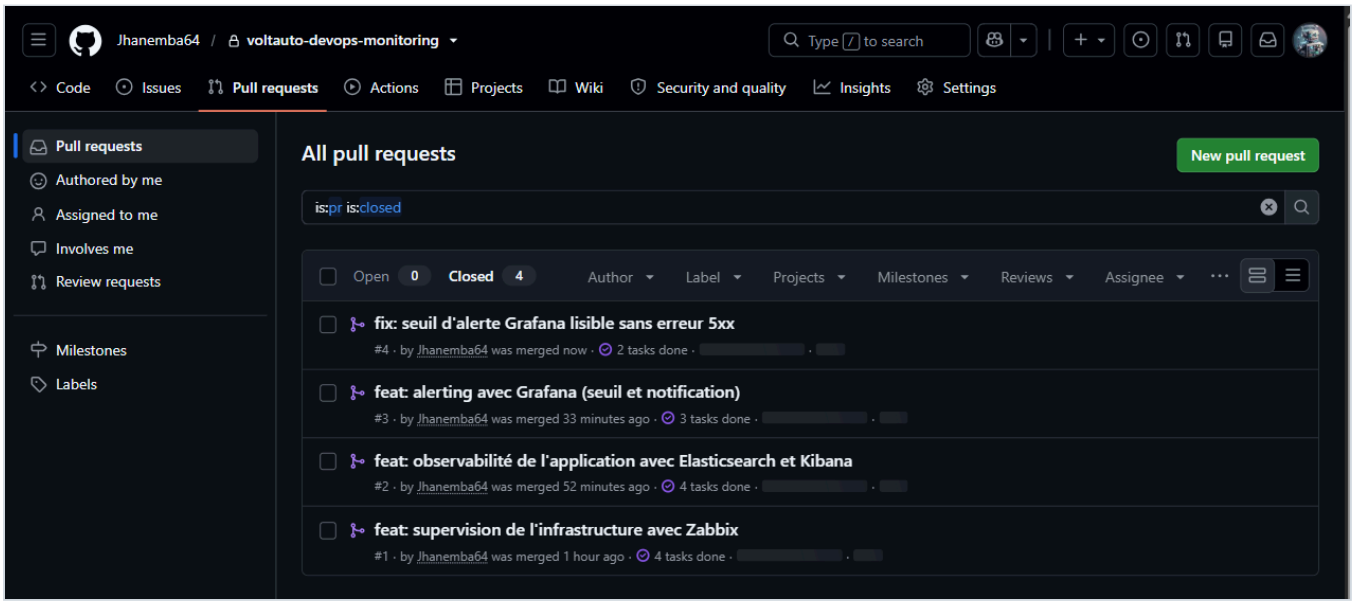


Figure 28 — Les quatre pull requests fusionnées (3 évolutions et 1 correctif)

9. Difficultés rencontrées et solutions

Problème	Cause	Solution
`localhost:80` affiche une page Apache inconnue	un service WSL écoute sur `::1:80`	tests du backend via `127.0.0.1:80`
Zabbix remonte tous les conteneurs de la machine	template Docker sans filtre	macro de filtre posée par `zabbix-init` + nom de projet compose fixé
Filebeat n'envoie aucun log	les conditions d'autodiscover sur les labels compose ne sont jamais vraies, et le `$` d'une expression régulière est absorbé par la configuration	filtre sur `docker.container.name`, diagnostiqué avec un banc de test Filebeat en sortie console
Le découpage du backend ne s'applique pas	les labels sont sous `docker.container.labels`, pas `container.labels`	condition corrigée
Import Kibana : erreur 500	objets écrits à la main sans version : Kibana leur applique d'anciennes migrations Lens	versions de schéma indiquées, puis export canonique depuis Kibana
Annotation Grafana non interprétée	`$$` n'est pas ré-échappé dans les fichiers d'alerting	`{{ .Values.B.Value }}`, sans `$`
Liens des notifications en `localhost:3000`	URL publique de Grafana non définie	`GF_SERVER_ROOT_URL=http://localhost:3001`
Seuil écrasé en bas du graphique Grafana au repos	sans erreur, l'axe passe en échelle 0–100	`custom.axisSoftMax: 10` (PR #4)
Modification du dashboard Grafana non prise en compte	fichier provisionné non relu à chaud depuis le montage Windows	redémarrage de `grafana` après chaque modification
Message de commit abîmé (apostrophe perdue)	guillemets du shell	commits refaits et `push --force-with-lease`, **avant** l'ouverture de la PR

10. Limites et pistes d'amélioration

- **Sécurité** : configuration de laboratoire. La sécurité Elasticsearch est désactivée (ports liés à `127.0.0.1`) et le compte Zabbix `Admin / zabbix` garde son mot de passe par défaut. Pour la production, il faudrait activer TLS et l'authentification, puis gérer les secrets dans un coffre.
- **Notifications** : le webhook simule l'outil de messagerie. Pour un vrai usage, il suffit de remplacer son URL par celle d'un webhook Slack ou Teams, ou de configurer un serveur SMTP.
- **Frontend** : il tourne en serveur de développement, comme dans le projet d'origine. Une image de production (build statique servi par nginx) serait plus légère.
- **Unification** : le plugin Zabbix de Grafana permettrait de regrouper métriques d'infrastructure et métriques applicatives dans le même outil d'alerting.
- **CI** : une GitHub Action pourrait valider `docker compose config`, le JSON des tableaux de bord et la syntaxe des scripts à chaque *pull request*.